

Polynomial-Time Reduction and NP-Completeness

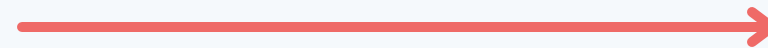
Concepts, Examples, Applications and Modern Enhancements

Why do we study computational complexity?

Complexity asks a practical question: what remains possible when the input becomes large?

SCALES WELL

10 tasks
Small input
manageable
minutes



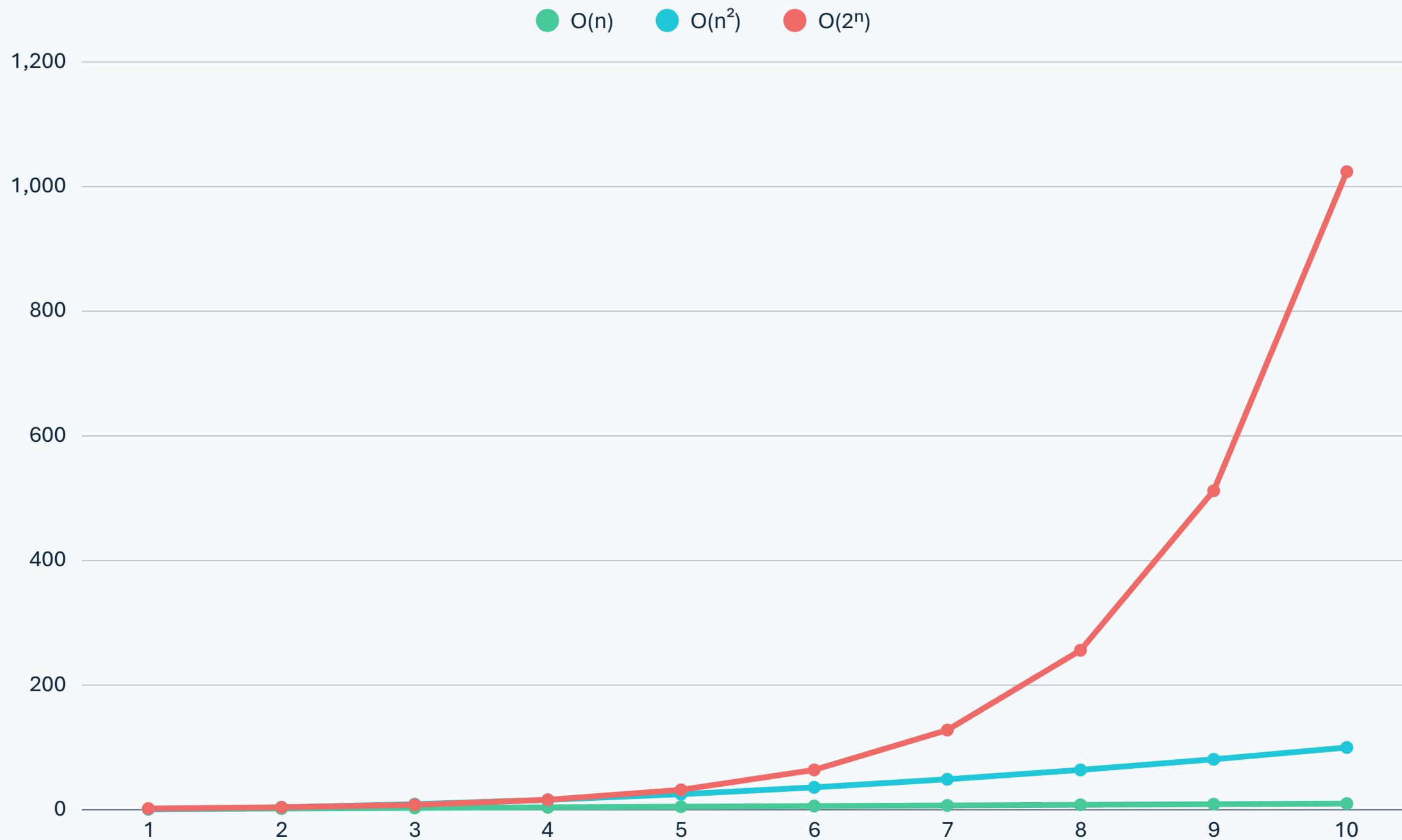
BECOMES INFEASIBLE

10^6 choices
Large input
combinatorial explosion
years

Complexity theory classifies difficulty • guides algorithm choice • exposes limits

Understanding time complexity

Growth rate comparison



$O(n)$

Direct growth

$O(n^2)$, $O(n^3)$

Polynomial — feasible for many inputs

$O(2^n)$

Exponential — quickly impractical

P and NP

P

SOLVE in polynomial time

Examples: sorting, shortest path



NP

VERIFY in polynomial time

Example: check a completed Sudoku

solve $\neq$ verify

What is polynomial time?

Runtime is bounded by n^k for some constant k .

SORTING

$O(n \log n)$

MATRIX MULTIPLICATION

$O(n^3)$

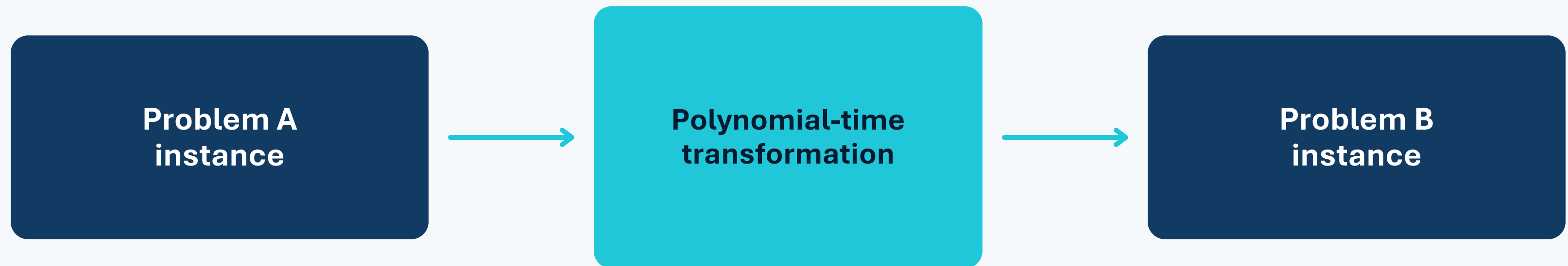
EXPONENTIAL

$O(2^n)$

Polynomial does not always mean fast — but it scales far better than exponential growth.

Polynomial-time reduction

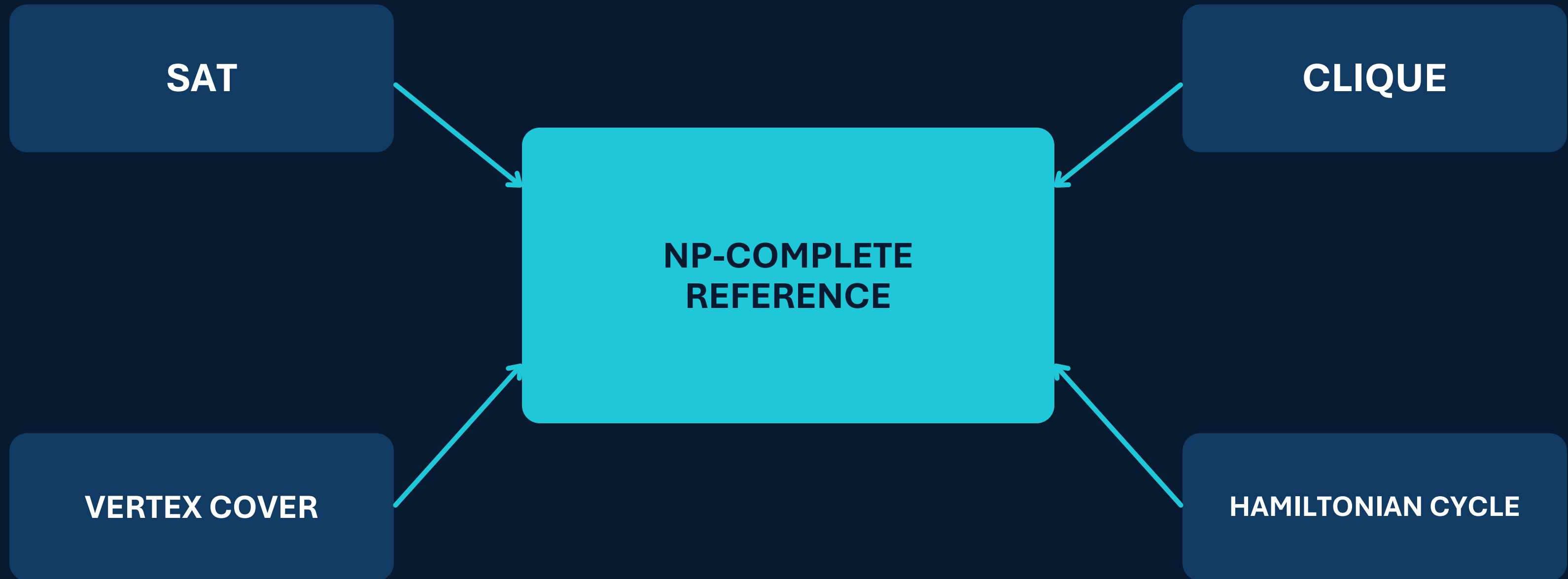
$A \leq_p B$ means: transform every instance of A into B in polynomial time.



If B is efficient, then A can be solved through B.

PURPOSE • compare difficulty • prove hardness • transfer knowledge

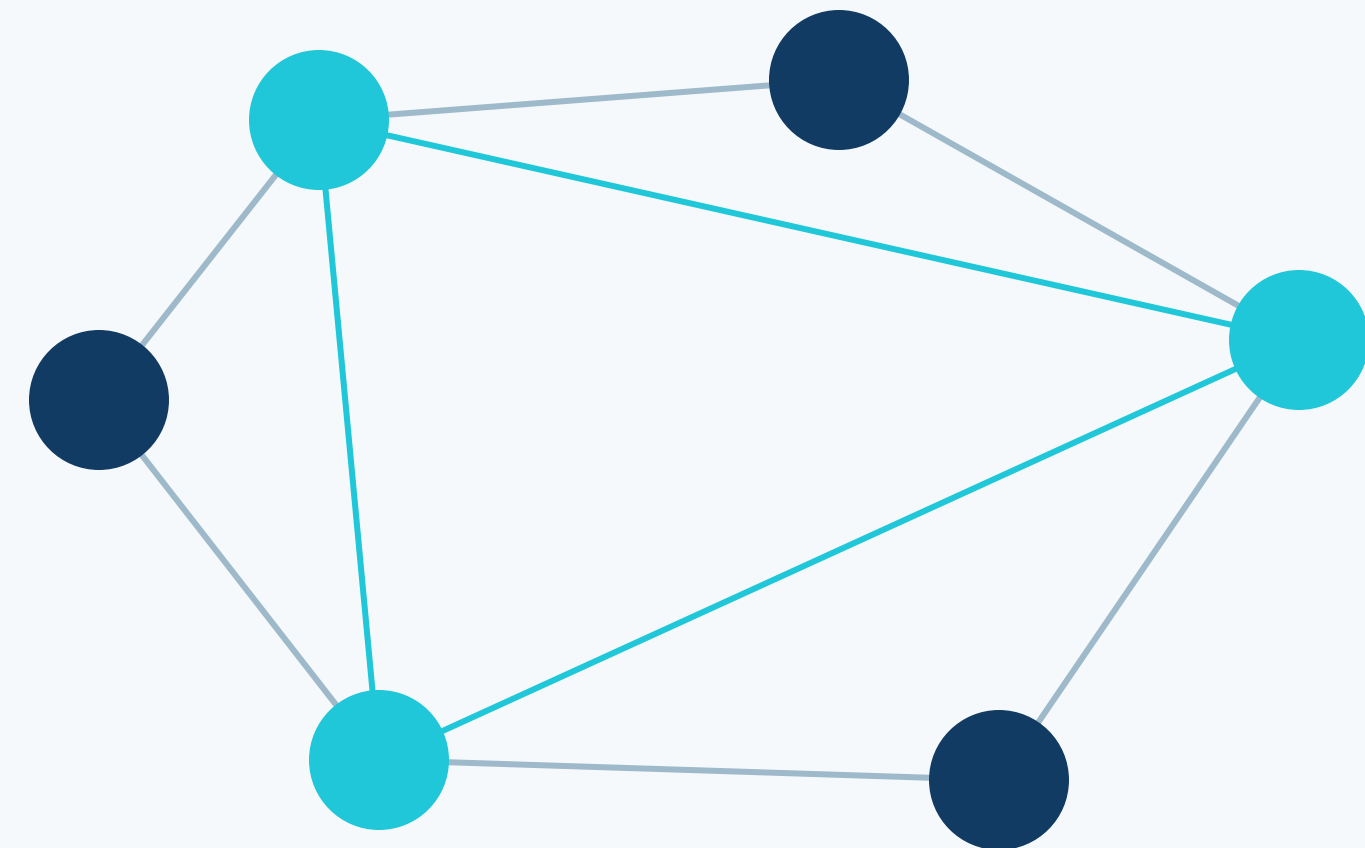
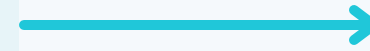
Why are reductions important?



A reduction is a difficulty-preserving bridge — not an algorithmic shortcut.

3-SAT → CLIQUE

$$\begin{aligned} & (x_1 \vee \neg x_2 \vee x_3) \\ & \wedge (\neg x_1 \vee x_2 \vee x_4) \\ & \wedge (x_2 \vee \neg x_3 \vee \neg x_4) \end{aligned}$$



Create one vertex per literal occurrence. Connect compatible literals from different clauses.

A clique of size 3 selects one consistent true literal from each clause.

What is NP-Complete?

1

IN NP

A proposed solution can be verified in polynomial time.

2

NP-HARD

Every problem in NP reduces to it in polynomial time.

NP-COMPLETE = IN NP + NP-HARD

The hardest decision problems inside NP

Famous NP-Complete problems

SAT

Satisfy a Boolean formula

3-SAT

Three literals per clause

CLIQUE

Find a complete subgraph

VERTEX COVER

Cover every graph edge

HAMILTONIAN CYCLE

Visit each vertex once

TSP • decision

Tour within a cost bound

P vs NP vs NP-Hard vs NP-Complete

CLASS	DEFINITION	POLY-TIME SOLUTION?	EXAMPLE
P	Solve + verify	Yes	Shortest path
NP	Verify	Unknown	Factor decision
NP-Hard	At least NP-hard	Not required	TSP optimization
NP-Complete	Verify + hardest in NP	Unknown	3-SAT

OPEN PROBLEM • Whether $P = NP$ remains unknown.

Where hard problems appear

SCHEDULING

Jobs, exams, factories

CIRCUIT DESIGN

Placement and verification

ROUTING

Deliveries and logistics

CYBERSECURITY

Detection and attack paths

NETWORK DESIGN

Topology and resilience

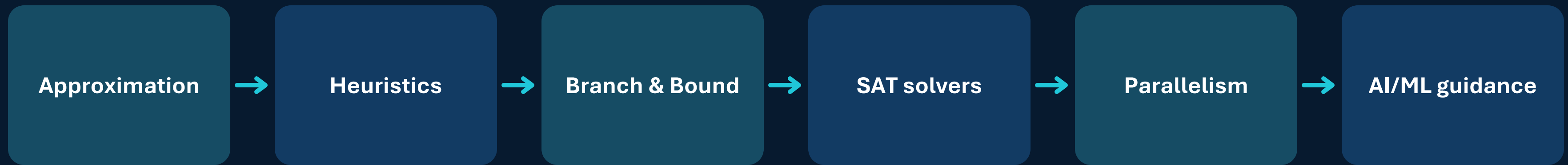
CLOUD OPTIMIZATION

Workload placement

RESOURCE ALLOCATION

People, machines, cloud

Modern techniques for NP-Complete problems



These methods improve practical outcomes — they do not establish polynomial-time solutions for NP-Complete problems.

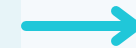
Goal: exact answers on smaller instances, or high-quality answers under real constraints.

AI/ML + NP-Complete problems

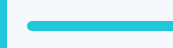
CASE • DELIVERY ROUTE OPTIMIZATION

Traditional search

Huge candidate space
High evaluation cost
Many unpromising branches



**ML model
ranks
candidates**



AI-guided search

Predict promising candidates
Prioritize useful branches
Learn from prior instances

AI guides the search. It does not prove $P = NP$.

Future scope

ESTABLISHED →

Advanced SAT
solvers

Hybrid
optimization

Distributed
computing

AI-assisted
search

Quantum
potential

SPECULATIVE →

Progress may transform what is practical — without changing worst-case complexity.

Key takeaways

P

efficiently solvable

NP

efficiently verifiable

$\leq_p$

compares problem difficulty

NP-C

hardest problems in NP

MODERN METHODS

good practical solutions

P = NP?

still open

References + viva questions

REFERENCES

Cormen et al. — Introduction to Algorithms

Arora & Barak — Computational Complexity

Stanford Encyclopedia of Philosophy

ACM / IEEE educational resources

VIVA • QUICK CHECK

- 1 What does NP mean? — Verifiable in polynomial time.
- 2 Define $\leq_p$. — Transform A to B in polynomial time.
- 3 NP-Complete? — In NP and NP-Hard.
- 4 Does AI solve NP-C in polynomial time? — No.
- 5 Why 3-SAT $\rightarrow$ CLIQUE? — Proves CLIQUE is at least as hard.
- 6 NP-Hard vs NP-C? — NP-C must also be in NP.
- 7 Why approximation? — Efficient near-optimal answers.
- 8 Is P = NP proven? — No; it remains open.